МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КУБГУ»)

Кафедра вычислительных технологий

Отчёт

по лабораторной работе №4

по дисциплине «Методы разработки трансляторов»

на тему: «Построение синтаксического анализатора»

Работу выполнил

студент группы

ФИ 31/2

Костров А.А.

_____

(подпись)

Преподаватель:

Усов П.Е.

_____

(подпись)

Краснодар
2026

Цель работы

Изучить принципы синтаксического анализа исходного кода, освоить метод рекурсивного спуска как способ разбора «сверху-вниз». Реализовать синтаксический анализатор языка программирования R, способный обнаруживать ошибки в основных синтаксических конструкциях, с графическим интерфейсом и наглядным отображением результатов.

Задачи

1. Изучить теоретические основы метода рекурсивного спуска, понятия нетерминала, фразы и цели разбора.

2. Разработать грамматику для основных конструкций языка R (условные операторы, циклы, функции, выражения).

3. Реализовать синтаксический анализатор на языке Python.

* Каждому нетерминальному символу грамматики ставится в соответствие отдельная процедура.

* Обеспечить обработку ошибочных ситуаций с выводом информативных сообщений (номер строки и столбца).

* Реализовать механизмы восстановления после ошибок для продолжения анализа.

4. Разработать графический интерфейс для анализатора.

* Поле ввода исходного кода.

* Панель результатов с отсортированным списком ошибок.

* Подсветка ошибочных строк непосредственно в исходном коде.

* Отображение статистики по типам найденных ошибок.

Теоретическая часть

Синтаксический анализ является вторым этапом работы компилятора и проверяет исходную программу на соответствие грамматике языка. В данной работе используется метод рекурсивного спуска, относящийся к классу нисходящих методов разбора.

Основная идея метода заключается в построении дерева разбора начиная с корня (начального символа грамматики <программа>) и продвижении слева направо. Ключевые понятия метода:

Цель — нетерминальный символ, который в данный момент является концевым в строящемся дереве разбора. Задача анализатора — найти вывод из этой цели.

Фраза — самая левая подцепочка входной программы, выводимая из текущей цели.

Каждому нетерминалу грамматики соответствует отдельная семантическая процедура (функция). Многократный вызов этих процедур в процессе разбора дал методу название «рекурсивный спуск». Процедура для нетерминала U анализирует входной поток, сравнивает лексемы с правыми частями правил для U, выбирает подходящее правило и, при необходимости, вызывает процедуры для других нетерминалов, входящих в это правило. Таким образом, анализ «спускается» от сложных конструкций к простым (терминалам).

Грамматика анализируемого подмножества языка R, положенная в основу анализатора:

* <программа> ::= { <оператор> }

* <оператор> ::= <присваивание> | <выражение> | <if> | <while> | <for> | <function> | <return> | { <блок> }

* <блок> ::= { { <оператор> } }

* <if> ::= if (<выражение>) <оператор> [else <оператор>]

* <while> ::= while (<выражение>) <оператор>

* <function> ::= function ([<параметры>]) <оператор>

* <выражение> ::= <унарное> { <бинарный_оператор> <унарное> }

Ход работы

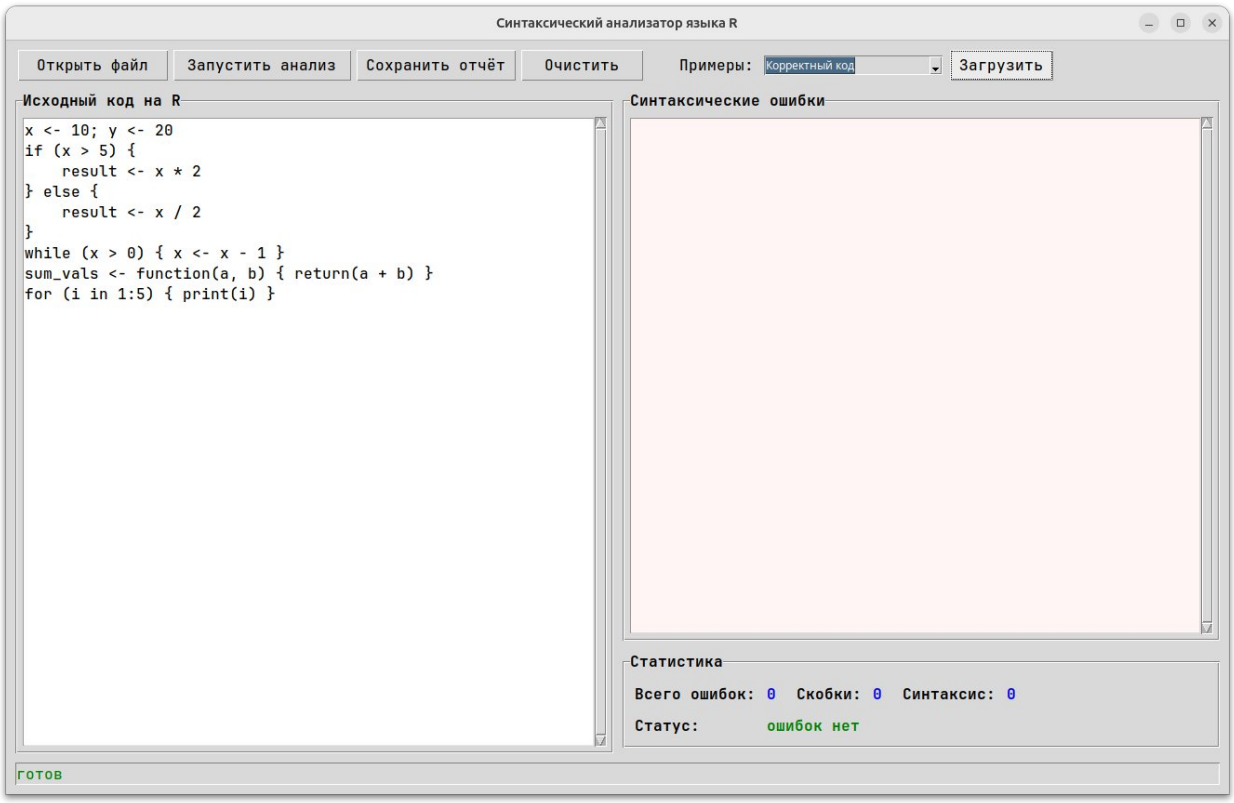
1. **Создание базовой структуры.** Был разработан класс RSyntaxAnalyzer, реализующий логику метода рекурсивного спуска. Токены из лексического анализатора (lexer.py) последовательно считываются и обрабатываются.

2. **Реализация процедур для нетерминалов.** Были написаны и отлажены основные функции-процедуры: _parse_program, _parse_statement, _parse_block, _parse_if, _parse_while, _parse_for, _parse_function_def, _parse_expression, _parse_assignment_or_expr.

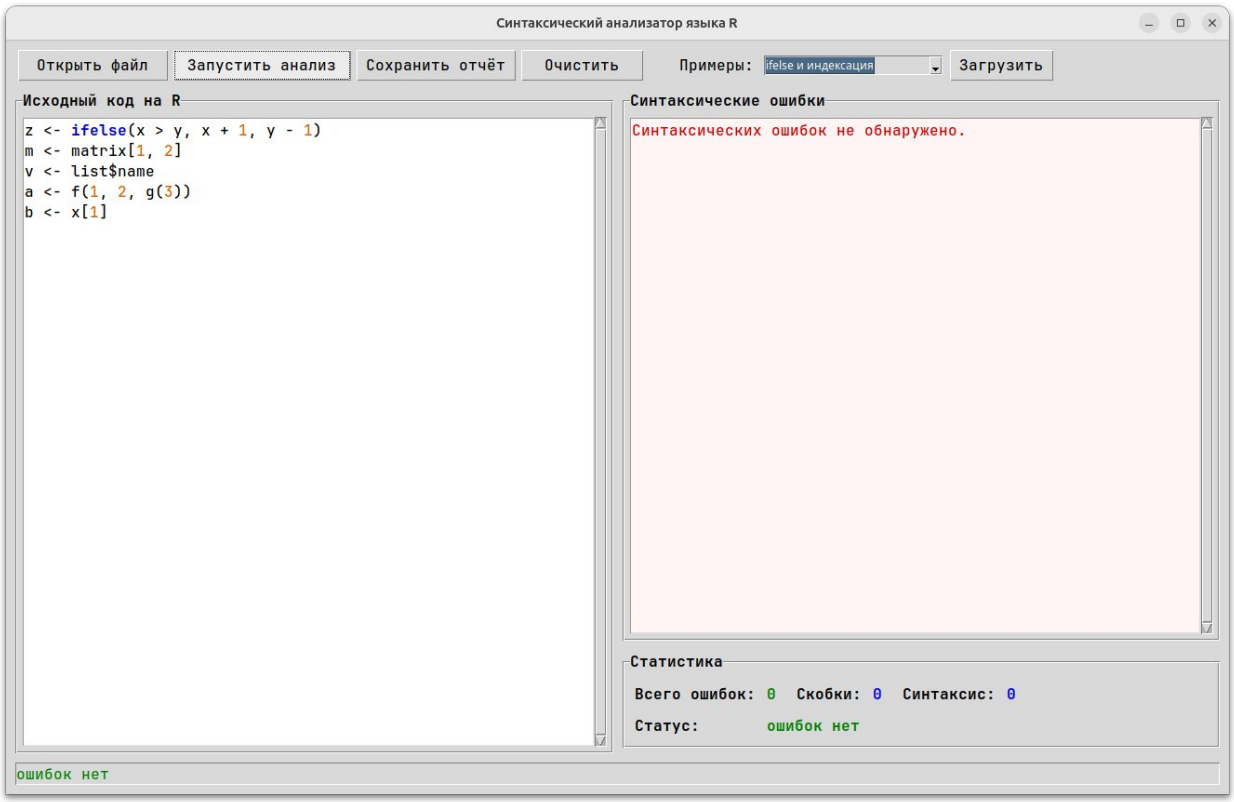
3. **Сортировка и отображение ошибок.** Для удобства пользователя все найденные ошибки сортируются по номеру строки и колонки. Финальные ошибки (незакрытые конструкции) выводятся в конце. В графическом интерфейсе ошибки группируются по категориям: "Синтаксические ошибки", "Ошибки скобок", "Незакрытые конструкции".

Описание программы

Программа представляет собой приложение с графическим интерфейсом, построенным с использованием библиотеки Tkinter.



Интерфейс разделен на две основные панели.

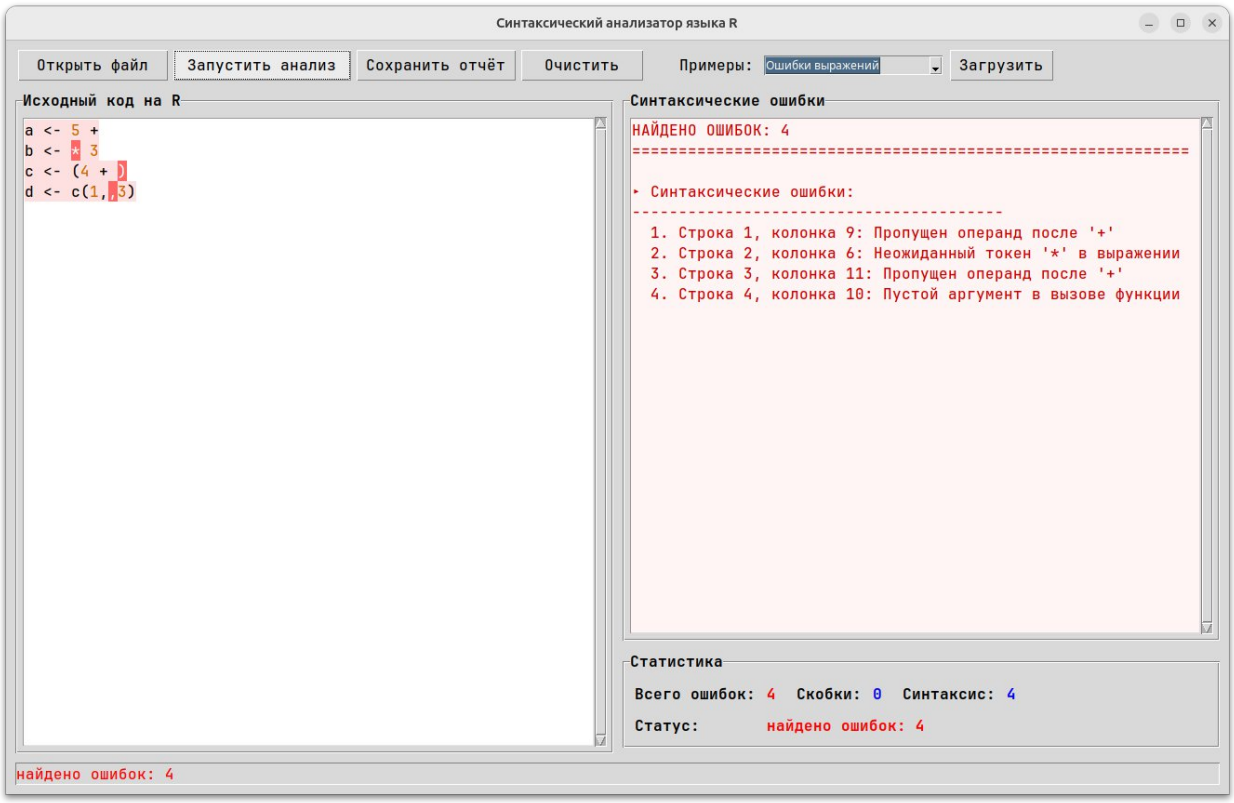


* **Левая панель ("Исходный код на R"):** содержит текстовый редактор для ввода исходного кода. Осуществляется базовая синтаксическая подсветка (ключевые слова, числа, строки, комментарии). При анализе строки с ошибками подсвечиваются красным фоном.

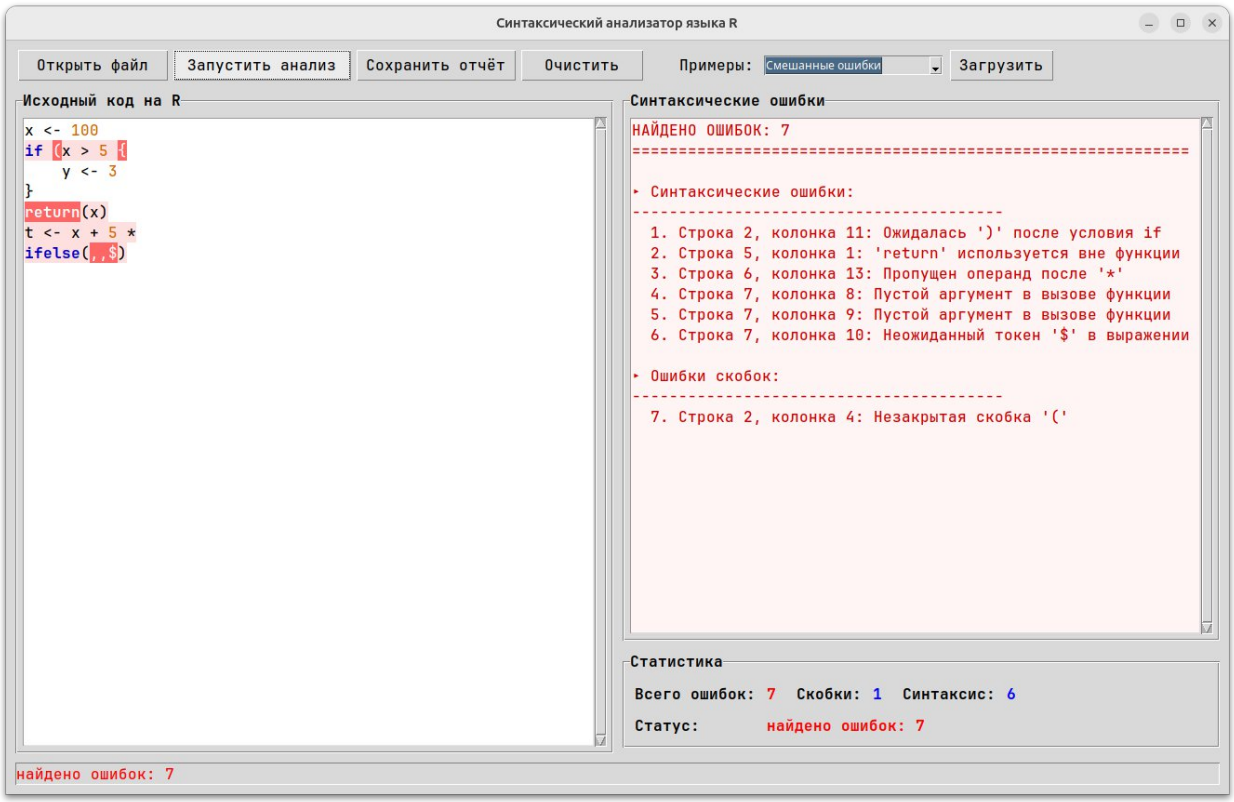
* **Правая панель:** содержит два блока.

  * **"Синтаксические ошибки":** выводит отсортированный и сгруппированный список найденных ошибок с указанием строки, колонки и описания.

  * **"Статистика":** отображает общее количество ошибок, а также количество ошибок по категориям.



В верхней части окна находится панель инструментов с кнопками "Открыть файл", "Запустить анализ", "Сохранить отчёт", "Очистить", а также выпадающий список с заранее заготовленными примерами для тестирования.



Полный код итоговой программы представлен ниже.

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
from dataclasses import dataclass, field
from typing import List, Optional, Tuple
import re
import datetime

from lexer import RLexer, LexemType, Token

@dataclass(order=True)
class SyntaxError:
    sort_line: int = field(init=False, default=0)
    sort_column: int = field(init=False, default=0)
    line: int = 0
    column: int = 0
    message: str = ""
    token_value: str = ""
    error_type: str = "syntax"

    def __post_init__(self):
        self.sort_line = self.line if self.line > 0 else 999999
        self.sort_column = self.column

    def __str__(self):
        if self.line > 0:
            return f"Строка {self.line}, колонка {self.column}: {self.message}"
        return f"Финальная проверка: {self.message}"

class ParseContext:
    def __init__(self):
        self.in_function: int = 0
        self.in_loop: int = 0
        self.control_stack: List[str] = []

class RSyntaxAnalyzer:

    CONSTANT_KEYWORDS = {'TRUE', 'FALSE', 'NULL', 'NA', 'Inf', 'NaN'}
    ASSIGN_OPS = {'←', '<<-', '->', '->>', '=', ':='}
    CONTROL_KEYWORDS = {'if', 'else', 'while', 'for', 'repeat', 'function', 'return', 'next', 'break'}

    ALL_OPS = {
        '+', '-', '*', '/', '^', '**', '%%', '%/%',
        '<', '>', '≤', '≥', '==', '≠',
        '&', '&&', '|', '||', '!',
        '$', '@', '~', ':'
    }

    PARAM_STOP_TOKENS = {'{', '}', '\n', ';', 'if', 'while', 'for', 'function', 'repeat', 'return'}
    EXPR_STOP_TOKENS = {';', '}', ')', ']', ',', '\n', 'else', '{'}
    STMT_RECOVERY = {';', '}', '\n'}

    def __init__(self):
        self.tokens: List[Token] = []
        self.pos: int = 0
        self.errors: List[SyntaxError] = []
        self.ctx: ParseContext = ParseContext()
        self._current: Optional[Token] = None
        self.bracket_stack: List[Tuple[str, int, int]] = []

    def reset(self):
        self.tokens = []
        self.pos = 0
        self.errors = []
        self.ctx = ParseContext()
        self._current = None
        self.bracket_stack = []

    def analyze(self, tokens: List[Token]) -> Tuple[bool, List[SyntaxError]]:
        self.reset()
        self.tokens = [t for t in tokens if t.lex_type != LexemType.COMMENT]

        if not self.tokens:
            return True, []
```

```
        self._advance()
        self._parse_program()

        self._final_bracket_check()
        self._check_unclosed_constructs()

        self.errors.sort(key=lambda e: (e.sort_line, e.sort_column))

        return len(self.errors) == 0, self.errors

    def _advance(self) -> Optional[Token]:
        while self.pos < len(self.tokens):
            tok = self.tokens[self.pos]
            self.pos += 1
            if tok.value in (' ', '\t', '\r'):
                continue
            self._current = tok
            return tok
        self._current = None
        return None

    def _peek(self, offset: int = 1) -> Optional[Token]:
        saved_pos = self.pos
        saved_cur = self._current

        count = 0
        search_pos = saved_pos
        result = None

        while search_pos < len(self.tokens):
            t = self.tokens[search_pos]
            search_pos += 1
            if t.value in (' ', '\t', '\r'):
                continue
            count += 1
            if count == offset:
                result = t
                break

        self.pos = saved_pos
        self._current = saved_cur
        return result

    def _current_value(self) -> str:
        return str(self._current.value) if self._current else "EOF"

    def _is_identifier(self) -> bool:
        if not self._current:
            return False
        if self._current.lex_type == LexemType.IDENTIFIER:
            return True
        if self._current.lex_type == LexemType.KEYWORD:
            return self._current.value not in self.CONTROL_KEYWORDS
        return False

    def _is_number_or_string(self) -> bool:
        if not self._current:
            return False
        return self._current.lex_type in (LexemType.NUMBER, LexemType.STRING)

    def _add_error(self, msg: str, token: Optional[Token] = None):
        t = token or self._current
        if t:
            self.errors.append(SyntaxError(t.line, t.column, msg, t.value, "syntax"))
        else:
            self.errors.append(SyntaxError(0, 0, msg, "", "syntax"))

    def _track_bracket(self, bracket: str, line: int, col: int):
        pairs = {'(': ')', '[': ']', '{': '}', '[[': ']]'}
        closing = {')': '(', ']': '[', '}': '{', ']]': '[['}

        if bracket in pairs:
            self.bracket_stack.append((bracket, line, col))
        elif bracket in closing:
            expected_open = closing[bracket]
            if not self.bracket_stack:
```

```
                self.errors.append(SyntaxError(line, col, f"Лишняя закрывающая скобка '{bracket}'", bracket,
"bracket"))
            else:
                open_br, open_line, open_col = self.bracket_stack.pop()
                if open_br ≠ expected_open:
                    self.errors.append(SyntaxError(
                        line, col,
                        f"Несоответствие скобок: открыта '{open_br}', закрыта '{bracket}'",
                        bracket, "bracket"
                    ))

    def _skip_until_match(self, targets: set):
        depth = 0
        while self._current:
            v = self._current.value
            if v in ('(', '[', '{'):
                depth += 1
            elif v in (')', ']', '}'):
                if depth == 0:
                    break
                depth -= 1
            if depth ≤ 0 and v in targets:
                break
            self._advance()

    def _safe_skip_to_statement(self):
        block_depth = 0
        paren_depth = 0
        bracket_depth = 0

        while self._current:
            v = self._current.value

            if v == '{':
                block_depth += 1
            elif v == '}':
                block_depth -= 1
                if block_depth < 0:
                    block_depth = 0
            elif v == '(':
                paren_depth += 1
            elif v == ')':
                paren_depth -= 1
                if paren_depth < 0:
                    paren_depth = 0
            elif v == '[':
                bracket_depth += 1
            elif v == ']':
                bracket_depth -= 1
                if bracket_depth < 0:
                    bracket_depth = 0

            if block_depth ≤ 0 and paren_depth ≤ 0 and bracket_depth ≤ 0:
                if v in ('if', 'while', 'for', 'repeat', 'function', 'return', 'next', 'break'):
                    break
                if v in (';', '\n', '}'):
                    self._advance()
                    break

            self._advance()

    def _parse_program(self):
        while self._current is not None:
            self._parse_statement()
            while self._current and self._current.value in (';', '\n'):
                self._advance()

    def _parse_statement(self):
        if self._current is None:
            return

        val = self._current_value()

        if val == 'if':
            self._parse_if()
        elif val == 'while':
            self._parse_while()
```

```
        elif val == 'for':
            self._parse_for()
        elif val == 'repeat':
            self._parse_repeat()
        elif val == 'function':
            self._parse_function_def()
        elif val == 'return':
            self._parse_return()
        elif val in ('next', 'break'):
            self._parse_next_break(val)
        elif val == '{':
            self._parse_block()
        elif val == '}':
            self._add_error("Лишняя закрывающая скобка '}'")
            self._track_bracket('}', self._current.line, self._current.column)
            self._advance()
        elif self._is_identifier():
            self._parse_assignment_or_expr()
        elif self._is_number_or_string() or val == '(':
            self._parse_expression()
        elif val in (';', '\n'):
            self._advance()
        else:
            self._add_error(f"Неожиданный токен '{val}' в начале оператора")
            self._safe_skip_to_statement()

    def _parse_block(self):
        if not self._current or self._current.value != '{':
            return

        self._track_bracket('{', self._current.line, self._current.column)
        self._advance()
        self.ctx.control_stack.append('{')

        while self._current and self._current.value != '}':
            self._parse_statement()
            while self._current and self._current.value in (';', '\n'):
                self._advance()

        if self._current and self._current.value == '}':
            self._track_bracket('}', self._current.line, self._current.column)
            self._advance()
            if self.ctx.control_stack and self.ctx.control_stack[-1] == '{':
                self.ctx.control_stack.pop()
        else:
            self._add_error("Незакрытый блок: ожидалась '}'")

    def _parse_if(self):
        if_token = self._current
        self._advance()
        self.ctx.control_stack.append('if')

        has_paren = False
        if self._current and self._current.value == '(':
            has_paren = True
            self._track_bracket('(', self._current.line, self._current.column)
            self._advance()
            self._parse_expression()
            if self._current and self._current.value == ')':
                self._track_bracket(')', self._current.line, self._current.column)
                self._advance()
            else:
                self._add_error("Ожидалась ')' после условия if")
                self._skip_until_match({')', '{', '\n', ';', 'else'})
                if self._current and self._current.value == ')':
                    self._track_bracket(')', self._current.line, self._current.column)
                    self._advance()
        else:
            self._add_error("Ожидалась '(' после if")

        if self._current and self._current.value == '{':
            self._parse_block()
        elif self._current and self._current.value not in (';', '\n', 'else', '}', ')'):
            self._parse_statement()

        if self._current and self._current.value == 'else':
            self._advance()
```

```
        if self._current and self._current.value == 'if':
            self._parse_if()
        elif self._current and self._current.value == '{':
            self._parse_block()
        else:
            self._parse_statement()

    if self.ctx.control_stack and self.ctx.control_stack[-1] == 'if':
        self.ctx.control_stack.pop()

def _parse_while(self):
    while_token = self._current
    self._advance()
    self.ctx.in_loop += 1
    self.ctx.control_stack.append('while')

    if self._current and self._current.value == '(':
        self._track_bracket('(', self._current.line, self._current.column)
        self._advance()
        self._parse_expression()
        if self._current and self._current.value == ')':
            self._track_bracket(')', self._current.line, self._current.column)
            self._advance()
        else:
            self._add_error("Ожидалась ')' после условия while")
            self._skip_until_match({')', '{', '\n', ';'})
            if self._current and self._current.value == ')':
                self._track_bracket(')', self._current.line, self._current.column)
                self._advance()
    else:
        self._add_error("Ожидалась '(' после while")

    if self._current and self._current.value == '{':
        self._parse_block()
    elif self._current and self._current.value not in (';', '\n', '}', ')'):
        self._parse_statement()

    self.ctx.in_loop -= 1
    if self.ctx.control_stack and self.ctx.control_stack[-1] == 'while':
        self.ctx.control_stack.pop()

def _parse_for(self):
    self._advance()
    self.ctx.in_loop += 1
    self.ctx.control_stack.append('for')

    if self._current and self._current.value == '(':
        self._track_bracket('(', self._current.line, self._current.column)
        self._advance()
    else:
        self._add_error("Ожидалась '(' после for")

    if self._is_identifier():
        self._advance()
    else:
        self._add_error("Ожидался идентификатор переменной цикла for")

    if self._current and self._current.value == 'in':
        self._advance()
    else:
        self._add_error("Ожидалось 'in' в цикле for")

    self._parse_expression()

    if self._current and self._current.value == ')':
        self._track_bracket(')', self._current.line, self._current.column)
        self._advance()
    else:
        self._add_error("Ожидалась ')' после выражения for")
        self._skip_until_match({')', '{', '\n', ';'})
        if self._current and self._current.value == ')':
            self._track_bracket(')', self._current.line, self._current.column)
            self._advance()

    if self._current and self._current.value == '{':
        self._parse_block()
    elif self._current and self._current.value not in (';', '\n', '}', ')'):
```

```
        self._parse_statement()

    self.ctx.in_loop -= 1
    if self.ctx.control_stack and self.ctx.control_stack[-1] == 'for':
        self.ctx.control_stack.pop()

def _parse_repeat(self):
    self._advance()
    self.ctx.in_loop += 1
    self.ctx.control_stack.append('repeat')

    if self._current and self._current.value == '{':
        self._parse_block()
    elif self._current and self._current.value not in (';', '\n', '}', ')'):
        self._parse_statement()

    self.ctx.in_loop -= 1
    if self.ctx.control_stack and self.ctx.control_stack[-1] == 'repeat':
        self.ctx.control_stack.pop()

def _parse_function_def(self):
    func_token = self._current
    self._advance()
    self.ctx.in_function += 1
    self.ctx.control_stack.append('function')

    if self._current and self._current.value == '(':
        self._track_bracket('(', self._current.line, self._current.column)
        self._advance()

        while self._current and self._current.value not in (')', '{', '}'):
            if self._current.value in self.PARAM_STOP_TOKENS:
                self._add_error(f"Неожиданный токен '{self._current.value}' в параметрах функции")
                break

            if self._current.value == ',':
                self._advance()
                continue

            if self._current.value == '...':
                self._advance()
                continue

            if self._is_identifier():
                self._advance()
                if self._current and self._current.value == '=':
                    self._advance()
                    self._parse_expression()
                continue

            self._add_error(f"Неожиданный токен '{self._current.value}' в параметрах функции")
            self._advance()

        if self._current and self._current.value == ')':
            self._track_bracket(')', self._current.line, self._current.column)
            self._advance()
        else:
            self._add_error("Ожидалась ')' после параметров функции")
            self._skip_until_match({')', '{', '\n'})
            if self._current and self._current.value == ')':
                self._track_bracket(')', self._current.line, self._current.column)
                self._advance()
    else:
        self._add_error("Ожидалась '(' после 'function'")

    if self._current and self._current.value == '{':
        self._parse_block()
    elif self._current and self._current.value not in (';', '\n', '}', ')'):
        self._parse_statement()

    self.ctx.in_function -= 1
    if self.ctx.control_stack and self.ctx.control_stack[-1] == 'function':
        self.ctx.control_stack.pop()

def _parse_return(self):
    tok = self._current
    self._advance()
```

```
        if self.ctx.in_function ≤ 0:
            self._add_error("'return' используется вне функции", tok)
        if self._current and self._current.value not in (';', '}', '\n', ')'):
            self._parse_expression()

    def _parse_next_break(self, keyword: str):
        tok = self._current
        self._advance()
        if self.ctx.in_loop ≤ 0:
            self._add_error(f"'{keyword}' используется вне цикла", tok)

    def _parse_assignment_or_expr(self):
        saved_pos = self.pos
        saved_cur = self._current

        self._advance()

        if self._current and self._current.value in self.ASSIGN_OPS:
            self._advance()
            self._parse_expression()
            return

        if self._current and self._current.value in ('(', '[', '[[', '$', '@'):
            self.pos = saved_pos
            self._current = saved_cur
            self._parse_expression()
            return

        self.pos = saved_pos
        self._current = saved_cur
        self._parse_expression()

    def _parse_expression(self):
        self._parse_unary()

        while self._current:
            v = self._current.value

            if v in self.ALL_OPS and v not in ('!',) and not (v in ('+', '-') and self._is_unary_context()):
                op = v
                self._advance()

                if self._current is None or self._current.value in self.EXPR_STOP_TOKENS:
                    self._add_error(f"Пропущен операнд после '{op}'")
                    break

                self._parse_unary()
            elif v in ('$', '@'):
                self._advance()
                if self._is_identifier() or self._is_number_or_string():
                    self._advance()
                else:
                    self._add_error(f"Ожидалось имя после '{v}'")
                    break
            else:
                break

    def _is_unary_context(self) -> bool:
        if not self._current or self._current.value not in ('+', '-'):
            return False

        if self.pos ≤ 1:
            return True

        search_pos = self.pos - 2
        prev_token = None
        while search_pos ≥ 0:
            t = self.tokens[search_pos]
            search_pos -= 1
            if t.value in (' ', '\t', '\r', '\n'):
                continue
            prev_token = t
            break

        if prev_token is None:
            return True
```

```
        prev_val = prev_token.value
        return prev_val in (
            '(', '[', '{', ',', ';', '\n',
            '←', '⇐', '->', '->>', '=', ':=',
            '+', '-', '*', '/', '^', '**', '%%', '%/%',
            '<', '>', '⩽', '⩾', '⩵', '≠',
            '&', '&&', '|', '||', '!', '~', ':',
            'if', 'while', 'for', 'return', 'in'
        )

    def _parse_unary(self):
        if self._current and self._current.value in ('!', '+', '-') and self._is_unary_context():
            op = self._current.value
            self._advance()
            if self._current is None or self._current.value in self.EXPR_STOP_TOKENS:
                self._add_error(f"Пропущен операнд после '{op}'")
            else:
                self._parse_unary()
        else:
            self._parse_operand()
            self._parse_postfix()

    def _parse_operand(self):
        if self._current is None:
            self._add_error("Ожидался операнд, достигнут конец файла")
            return

        tok = self._current

        if self._is_number_or_string():
            self._advance()
            return

        if tok.lex_type == LexemType.KEYWORD and tok.value in self.CONSTANT_KEYWORDS:
            self._advance()
            return

        if self._is_identifier():
            self._advance()
            return

        if tok.value == '(':
            self._track_bracket('(', tok.line, tok.column)
            self._advance()
            self._parse_expression()
            if self._current and self._current.value == ')':
                self._track_bracket(')', self._current.line, self._current.column)
                self._advance()
            else:
                self._add_error("Ожидалась ')'")
                self._skip_until_match({')', ';', '}', '\n'})
                if self._current and self._current.value == ')':
                    self._track_bracket(')', self._current.line, self._current.column)
                    self._advance()
            return

        if tok.value == 'function':
            self._parse_function_def()
            return

        self._add_error(f"Неожиданный токен '{tok.value}' в выражении")
        self._advance()

    def _parse_postfix(self):
        while self._current:
            if self._current.value == '(':
                self._track_bracket('(', self._current.line, self._current.column)
                self._advance()
                self._parse_arg_list()
                if self._current and self._current.value == ')':
                    self._track_bracket(')', self._current.line, self._current.column)
                    self._advance()
                else:
                    self._add_error("Ожидалась ')' после аргументов функции")
                    self._skip_until_match({')', ';', '}', '\n'})
                    if self._current and self._current.value == ')':
                        self._track_bracket(')', self._current.line, self._current.column)
```

```
                    self._advance()
                break
        elif self._current.value == '[':
            self._parse_indexing_single()
        elif self._current.value == '[[':
            self._parse_indexing_double()
        elif self._current.value in ('$', '@'):
            op = self._current.value
            self._advance()
            if self._is_identifier() or self._is_number_or_string():
                self._advance()
            elif self._current and self._current.lex_type == LexemType.KEYWORD:
                self._advance()
            else:
                self._add_error(f"Ожидалось имя после '{op}'")
                break
        else:
            break

def _parse_arg_list(self):
    if self._current and self._current.value == ')':
        return

    while self._current:
        if self._current.value == ')':
            break

        if self._current.value == ',':
            self._add_error("Пустой аргумент в вызове функции")
            self._advance()
            continue

        self._parse_expression()

        if self._current and self._current.value == ',':
            self._advance()
        elif self._current and self._current.value == ')':
            break
        else:
            break

def _parse_indexing_single(self):
    tok = self._current
    self._track_bracket('[', tok.line, tok.column)
    self._advance()

    self._parse_arg_list()

    if self._current and self._current.value == ']':
        self._track_bracket(']', self._current.line, self._current.column)
        self._advance()
    else:
        self._add_error("Ожидался ']'")
        self._skip_until_match({']', ';', '}', '\n'})
        if self._current and self._current.value == ']':
            self._track_bracket(']', self._current.line, self._current.column)
            self._advance()

def _parse_indexing_double(self):
    tok = self._current
    self._track_bracket('[[', tok.line, tok.column)
    self._advance()

    self._parse_expression()

    if self._current and self._current.value == ']]':
        self._track_bracket(']]', self._current.line, self._current.column)
        self._advance()
    else:
        self._add_error("Ожидался ']]'")
        self._skip_until_match({']]', ';', '}', '\n'})

def _final_bracket_check(self):
    for bracket, line, col in self.bracket_stack:
        self.errors.append(SyntaxError(line, col, f"Незакрытая скобка '{bracket}'", bracket, "bracket"))

def _check_unclosed_constructs(self):
```

```
        names = {
            'function': 'определение функции',
            'if': 'условный оператор',
            'while': 'цикл while',
            'for': 'цикл for',
            'repeat': 'цикл repeat',
            '{': "блок '{'"
        }
        for item in self.ctx.control_stack:
            desc = names.get(item, f"конструкция '{item}'")
            self.errors.append(SyntaxError(0, 0, f"Незавершённая {desc}", "", "block"))


class SyntaxAnalyzerGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Синтаксический анализатор языка R")
        self.root.geometry("1600x950")

        from tkinter import font
        available = list(font.families())
        pref = ['JetBrains Mono', 'Consolas', 'Courier New', 'Courier', 'TkFixedFont']
        mono = 'TkFixedFont'
        for p in pref:
            if p in available:
                mono = p
                break

        self.font = (mono, 12)
        self.bold_font = (mono, 12, 'bold')
        self.small_font = (mono, 11)

        style = ttk.Style()
        style.configure('.', font=self.font)
        style.configure('TLabelframe.Label', font=self.bold_font)

        self.lexer = RLexer()
        self.analyzer = RSyntaxAnalyzer()

        self.setup_ui()
        self.setup_tags()
        self.load_example("Корректный код")

    def setup_ui(self):
        main = ttk.Frame(self.root, padding="10")
        main.grid(row=0, column=0, sticky="nsew")
        self.root.columnconfigure(0, weight=1)
        self.root.rowconfigure(0, weight=1)
        main.columnconfigure(0, weight=1)
        main.columnconfigure(1, weight=1)
        main.rowconfigure(1, weight=1)

        tb = ttk.Frame(main)
        tb.grid(row=0, column=0, columnspan=2, sticky="ew", pady=(0, 10))
        ttk.Button(tb, text="Открыть файл", command=self.open_file, width=15).pack(side=tk.LEFT, padx=3)
        ttk.Button(tb, text="Запустить анализ", command=self.run_analysis, width=18).pack(side=tk.LEFT, padx=3)
        ttk.Button(tb, text="Сохранить отчёт", command=self.save_report, width=16).pack(side=tk.LEFT, padx=3)
        ttk.Button(tb, text="Очистить", command=self.clear_all, width=12).pack(side=tk.LEFT, padx=3)

        ttk.Label(tb, text="  Примеры:").pack(side=tk.LEFT, padx=(15, 5))
        self.ex_var = tk.StringVar()
        examples = [
            "Корректный код",
            "Ошибки if-else",
            "Ошибки циклов",
            "Ошибки функций",
            "Ошибки скобок",
            "Ошибки выражений",
            "Смешанные ошибки",
            "ifelse и индексация"
        ]
        combo = ttk.Combobox(tb, textvariable=self.ex_var, values=examples, state="readonly", width=25)
        combo.pack(side=tk.LEFT, padx=5)
        ttk.Button(tb, text="Загрузить", command=self.load_selected_example, width=10).pack(side=tk.LEFT,
padx=3)

        left = ttk.LabelFrame(main, text="Исходный код на R", padding="5")
```

```
        left.grid(row=1, column=0, sticky="nsew", padx=(0, 5))
        left.columnconfigure(0, weight=1)
        left.rowconfigure(0, weight=1)
        self.code_text = scrolledtext.ScrolledText(left, wrap=tk.WORD, font=self.font, bg='white')
        self.code_text.grid(row=0, column=0, sticky="nsew")

        right = ttk.Frame(main)
        right.grid(row=1, column=1, sticky="nsew", padx=(5, 0))
        right.columnconfigure(0, weight=1)
        right.rowconfigure(0, weight=2)
        right.rowconfigure(1, weight=1)

        err_frame = ttk.LabelFrame(right, text="Синтаксические ошибки", padding="5")
        err_frame.grid(row=0, column=0, sticky="nsew", pady=(0, 5))
        err_frame.columnconfigure(0, weight=1)
        err_frame.rowconfigure(0, weight=1)
        self.errors_text = scrolledtext.ScrolledText(err_frame, wrap=tk.WORD, font=self.font, bg='#fff5f5',
fg='#cc0000')
        self.errors_text.grid(row=0, column=0, sticky="nsew")

        stats_frame = ttk.LabelFrame(right, text="Статистика", padding="10")
        stats_frame.grid(row=1, column=0, sticky="ew")

        self.stats_labels = {}

        ttk.Label(stats_frame, text="Всего ошибок:", font=self.bold_font).grid(row=0, column=0, sticky='w',
padx=(0, 10))
        self.stats_labels["Всего ошибок:"] = ttk.Label(stats_frame, text="0", font=self.bold_font,
foreground="blue")
        self.stats_labels["Всего ошибок:"].grid(row=0, column=1, sticky='w', padx=(0, 20))

        ttk.Label(stats_frame, text="Скобки:", font=self.bold_font).grid(row=0, column=2, sticky='w', padx=(0,
10))
        self.stats_labels["Скобки:"] = ttk.Label(stats_frame, text="0", font=self.bold_font, foreground="blue")
        self.stats_labels["Скобки:"].grid(row=0, column=3, sticky='w', padx=(0, 20))

        ttk.Label(stats_frame, text="Синтаксис:", font=self.bold_font).grid(row=0, column=4, sticky='w',
padx=(0, 10))
        self.stats_labels["Синтаксис:"] = ttk.Label(stats_frame, text="0", font=self.bold_font,
foreground="blue")
        self.stats_labels["Синтаксис:"].grid(row=0, column=5, sticky='w')

        ttk.Label(stats_frame, text="Статус:", font=self.bold_font).grid(row=1, column=0, sticky='w', padx=(0,
10), pady=(10, 0))
        self.stats_labels["Статус:"] = ttk.Label(stats_frame, text="Ошибок нет", font=self.bold_font,
foreground="green")
        self.stats_labels["Статус:"].grid(row=1, column=1, columnspan=5, sticky='w', pady=(10, 0))

        self.status_label = ttk.Label(main, text="Готов", relief=tk.SUNKEN, anchor=tk.W)
        self.status_label.grid(row=2, column=0, columnspan=2, sticky="ew", pady=(10, 0))

        self.ex_var.set("Корректный код")

    def setup_tags(self):
        self.code_text.tag_configure("keyword", foreground="#0000cc", font=self.bold_font)
        self.code_text.tag_configure("string", foreground="#008800")
        self.code_text.tag_configure("comment", foreground="#888888", font=(self.font[0], self.font[1],
'italic'))
        self.code_text.tag_configure("number", foreground="#cc6600")
        self.code_text.tag_configure("error_line", background="#ffe0e0")
        self.code_text.tag_configure("error_token", background="#ff6666", foreground="white")

    def highlight_syntax(self):
        for tag in ("keyword", "string", "comment", "number", "error_line", "error_token"):
            self.code_text.tag_remove(tag, 1.0, tk.END)

        code = self.code_text.get(1.0, tk.END)
        for m in re.finditer(r'#.*$', code, re.MULTILINE):
            self._tag_range("comment", m)
        for m in re.finditer(r'"[^"\\]*(?:\\.[^"\\]*)*"|\'[^\'\\]*(?:\\.[^\'\\]*)*\'', code):
            self._tag_range("string", m)
        kw =
r'\b(?:if|else|while|for|repeat|function|return|next|break|in|TRUE|FALSE|NULL|NA|Inf|NaN|ifelse)\b'
        for m in re.finditer(kw, code):
            self._tag_range("keyword", m)
        for m in re.finditer(r'\b\d+\.?\d*(?:[eE][+-]?\d+)?\b|\.\d+(?:[eE][+-]?\d+)?\b', code):
            self._tag_range("number", m)
```

```
def _tag_range(self, tag, match):
    self.code_text.tag_add(tag, f"1.0 + {match.start()} chars", f"1.0 + {match.end()} chars")

def highlight_errors(self, errors: List[SyntaxError]):
    self.code_text.tag_remove("error_line", 1.0, tk.END)
    self.code_text.tag_remove("error_token", 1.0, tk.END)
    lines_done = set()
    for e in errors:
        if e.line > 0 and e.line not in lines_done:
            self.code_text.tag_add("error_line", f"{e.line}.0", f"{e.line}.0 lineend")
            lines_done.add(e.line)
        if e.line > 0 and e.column > 0 and e.token_value:
            start = f"{e.line}.{e.column - 1}"
            end = f"{e.line}.{e.column - 1 + len(e.token_value)}"
            self.code_text.tag_add("error_token", start, end)

def run_analysis(self):
    code = self.code_text.get(1.0, tk.END)
    if not code.strip():
        messagebox.showwarning("Предупреждение", "Нет кода для анализа!")
        return

    self.status_label.config(text="Анализ...", foreground="orange")
    self.root.update()

    try:
        tokens = self.lexer.tokenize(code)
        success, errors = self.analyzer.analyze(tokens)

        self.highlight_syntax()
        self.highlight_errors(errors)
        self._display_errors(errors)

        bc = sum(1 for e in errors if e.error_type == "bracket")
        sc = len(errors) - bc

        self.stats_labels["Всего ошибок:"].config(
            text=str(len(errors)), foreground="red" if errors else "green"
        )
        self.stats_labels["Скобки:"].config(text=str(bc))
        self.stats_labels["Синтаксис:"].config(text=str(sc))

        status_text = "ошибок нет" if not errors else f"найдено ошибок: {len(errors)}"
        self.stats_labels["Статус:"].config(
            text=status_text, foreground="green" if not errors else "red"
        )
        self.status_label.config(text=status_text, foreground="green" if not errors else "red")

    except Exception as e:
        import traceback
        traceback.print_exc()
        messagebox.showerror("Ошибка", f"Ошибка при анализе:\n{str(e)}")
        self.status_label.config(text="Ошибка анализа", foreground="red")

def _display_errors(self, errors):
    self.errors_text.delete(1.0, tk.END)
    if not errors:
        self.errors_text.insert(1.0, "Синтаксических ошибок не обнаружено.\n")
        return

    bracket_errors = [e for e in errors if e.error_type == "bracket"]
    syntax_errors = [e for e in errors if e.error_type == "syntax"]
    block_errors = [e for e in errors if e.error_type == "block"]

    self.errors_text.insert(1.0, f"НАЙДЕНО ОШИБОК: {len(errors)}\n" + "=" * 60 + "\n\n")

    counter = 1

    if syntax_errors:
        self.errors_text.insert(tk.END, "▸ Синтаксические ошибки:\n" + "-" * 40 + "\n")
        for e in syntax_errors:
            self.errors_text.insert(tk.END, f"{counter:3d}. {e}\n")
            counter += 1
        self.errors_text.insert(tk.END, "\n")

    if bracket_errors:
```

```
            self.errors_text.insert(tk.END, "▸ Ошибки скобок:\n" + "-" * 40 + "\n")
            for e in bracket_errors:
                self.errors_text.insert(tk.END, f"{counter:3d}. {e}\n")
                counter += 1
            self.errors_text.insert(tk.END, "\n")

        if block_errors:
            self.errors_text.insert(tk.END, "▸ Незакрытые конструкции:\n" + "-" * 40 + "\n")
            for e in block_errors:
                self.errors_text.insert(tk.END, f"{counter:3d}. {e}\n")
                counter += 1

    def open_file(self):
        fn = filedialog.askopenfilename(filetypes=[("R files", "*.r"), ("All", "*.*")])
        if fn:
            with open(fn, 'r', encoding='utf-8') as f:
                self.code_text.delete(1.0, tk.END)
                self.code_text.insert(1.0, f.read())
            self.clear_results()

    def load_selected_example(self):
        ex = self.get_examples()
        sel = self.ex_var.get()
        if sel in ex:
            self.code_text.delete(1.0, tk.END)
            self.code_text.insert(1.0, ex[sel])
            self.clear_results()

    def load_example(self, name):
        self.code_text.delete(1.0, tk.END)
        self.code_text.insert(1.0, self.get_examples().get(name, ""))
        self.clear_results()

    def get_examples(self):
        return {
            "Корректный код": """x ← 10; y ← 20
if (x > 5) {
    result ← x * 2
} else {
    result ← x / 2
}
while (x > 0) { x ← x - 1 }
sum_vals ← function(a, b) { return(a + b) }
for (i in 1:5) { print(i) }""",
            "Ошибки if-else": """z ← 5
y ← 0
if (x > 5 { y ← 1 }
if (z < 0) { } else { } }""",
            "Ошибки циклов": """i ← 0
while i < 10 { i ← i + 1 }
for (j 1:10) { print(j) }
next
break""",
            "Ошибки функций": """return(5)
f ← function( { }
g ← function(x, y { }""",
            "Ошибки скобок": """x ← (1 + 2
y ← matrix[1, 2)
z ← (x + y))""",
            "Ошибки выражений": """a ← 5 +
b ← * 3
c ← (4 + )
d ← c(1,,3)""",
            "Смешанные ошибки": """x ← 100
if (x > 5 {
    y ← 3
}
return(x)
t ← x + 5 *
ifelse(,,$)""",
            "ifelse и индексация": """z ← ifelse(x > y, x + 1, y - 1)
m ← matrix[1, 2]
v ← list$name
a ← f(1, 2, g(3))
b ← x[1]"""
        }
```

```
    def clear_results(self):
        self.errors_text.delete(1.0, tk.END)
        self.code_text.tag_remove("error_line", 1.0, tk.END)
        self.code_text.tag_remove("error_token", 1.0, tk.END)
        for k in self.stats_labels:
            if k == "Статус:":
                self.stats_labels[k].config(text="ошибок нет", foreground="green")
            else:
                self.stats_labels[k].config(text="0", foreground="blue")
        self.status_label.config(text="готов", foreground="green")

    def clear_all(self):
        self.code_text.delete(1.0, tk.END)
        self.clear_results()

    def save_report(self):
        fn = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Текст", "*.txt")])
        if fn:
            with open(fn, 'w', encoding='utf-8') as f:
                f.write("ОТЧЁТ СИНТАКСИЧЕСКОГО АНАЛИЗА\n")
                f.write(f"Дата: {datetime.datetime.now()}\n")
                f.write("=" * 60 + "\n\n")
                f.write(self.errors_text.get(1.0, tk.END))
            messagebox.showinfo("Сохранено", f"Отчёт сохранён в {fn}")


def main():
    root = tk.Tk()
    app = SyntaxAnalyzerGUI(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

Структура программы

Программный код состоит из двух основных классов и модуля GUI.

1. Модуль syntax.py (синтаксический анализатор)

Содержит класс RSyntaxAnalyzer. Его ключевые атрибуты и методы:

* CONSTANT_KEYWORDS, ASSIGN_OPS, CONTROL_KEYWORDS: множества для распознавания типов токенов.

* analyze(tokens): главный метод, запускающий анализ и возвращающий список ошибок.

* _parse_program(), _parse_statement(), ..., _parse_expression(): семантические процедуры для нетерминалов грамматики.

* _add_error(msg, token): регистрирует синтаксическую ошибку.

* _skip_until_match(targets): вспомогательный метод для безопасного пропуска токенов при восстановлении после ошибки.

2. Класс SyntaxAnalyzerGUI

Обеспечивает графический интерфейс:

* setup_ui(): создает все виджеты окна.

* run_analysis(): запускает лексический (RLexer) и синтаксический (RSyntaxAnalyzer) анализ.

* _display_errors(errors): форматирует и выводит список ошибок в текстовое поле с группировкой.

* highlight_errors(errors): подсвечивает ошибочные строки и токены в редакторе кода.

Вывод

Разработанный синтаксический анализатор успешно справляется с задачей проверки кода на языке R на соответствие грамматике. Реализация метода рекурсивного спуска позволила создать ясную и расширяемую структуру, где каждая синтаксическая конструкция обрабатывается отдельной процедурой. Это делает код удобным для дальнейшей модификации и поддержки.